

Assignment: Advanced SQL Operations & Business Logic

Scenario: As the lead database developer, you are tasked with generating several critical reports for the executive team. Read the business requirements carefully and use your knowledge of Set Operators, JOINS, Subqueries, CTEs, and CASE statements to extract the correct data.

Question 1: Combining Sets (UNION)

The IT department needs a master list of employee numbers (`emp_no`) for a security audit. They need the IDs of anyone who has *ever* been a Department Manager, combined with anyone who has *ever* held the title of 'Technique Leader'. **Task:** Write a query using a Set Operator to combine these two lists into a single column. Ensure there are no duplicate employee numbers in the final output.

Question 2: Finding Differences (EXCEPT)

The promotions committee wants to identify employees who might be stuck in their roles. They want a list of employee numbers who have held the title of 'Engineer' but have *never* been promoted to 'Senior Engineer'. **Task:** Use the EXCEPT operator (or MINUS depending on your SQL dialect) against the `titles` table to find `emp_nos` that exist as 'Engineer' but do not exist as 'Senior Engineer'.

Question 3: The Missing Manager Audit (LEFT JOIN)

HR wants a complete list of all departments to verify their leadership. **Task:** Write a query that lists every `dept_name` from the `departments` table alongside the `first_name` and `last_name` of their current managers (`to_date = '9999-01-01'`). You must use a LEFT JOIN starting from the `departments` table so that if a department somehow has no active manager, the department name still appears in the report with NULL for the manager's name.

Question 4: Nested Lookups (IN Subquery)

The Sales Director wants a quick list of their current team, but they challenged you to do it *without* using any JOINS. **Task:** Write a query using uncorrelated subqueries (the IN operator) to find the `first_name` and `last_name` of all employees who *currently* work in the 'Sales' department. (Hint: You will need to nest a query inside a query inside a query: `Employees -> Dept_Emp -> Departments`).

Question 5: High-Performance Lookups (EXISTS)

Finance is running an audit and needs to flag highly compensated employees quickly without pulling in the entire salary history table. **Task:** Use the EXISTS operator to find the `first_name` and `last_name` of employees who have a *current* salary (`to_date = '9999-01-01'`) strictly greater than \$120,000.

Question 6: The Salary Brackets (CTE & CASE)

The accounting team is building a new dashboard and needs to classify current employees into salary bands. **Task:**

1. First, create a Common Table Expression (WITH statement) named `CurrentSalaries` that selects `emp_no` and `salary` where the `to_date` is active.
2. Next, write a main query that joins the `employees` table to your `CurrentSalaries` CTE.
3. Use a `CASE` statement in the `SELECT` clause to create a new column called `salary_band`:
 - If salary is greater than \$100,000 -> 'High Earner '
 - If salary is between \$70,000 and \$100,000 -> 'Mid Earner '
 - Otherwise -> 'Standard Earner '